

# Raport z Testów Systemu IO\_RNG

## System Testowania i Porównywania Generatorów Liczb Pseudolosowych

Projekt IO

January 15, 2026

## Contents

|          |                                             |          |
|----------|---------------------------------------------|----------|
| <b>1</b> | <b>Wprowadzenie</b>                         | <b>3</b> |
| 1.1      | Cel dokumentu . . . . .                     | 3        |
| 1.2      | Zakres testów . . . . .                     | 3        |
| 1.3      | Środowisko testowe . . . . .                | 3        |
| <b>2</b> | <b>Strategia Testowania</b>                 | <b>4</b> |
| 2.1      | Piramida testów . . . . .                   | 4        |
| 2.2      | Metodologia . . . . .                       | 4        |
| <b>3</b> | <b>Testy Jednostkowe</b>                    | <b>4</b> |
| 3.1      | Testy encji domenowych . . . . .            | 4        |
| 3.1.1    | Testowanie klasy RNG . . . . .              | 4        |
| 3.1.2    | Testowanie klasy TestResult . . . . .       | 4        |
| 3.2      | Testy repozytoriów . . . . .                | 5        |
| 3.2.1    | Testowanie DjangoRNGRepository . . . . .    | 5        |
| 3.3      | Testy runnerów . . . . .                    | 5        |
| 3.3.1    | Testowanie PythonRNGRunner . . . . .        | 5        |
| <b>4</b> | <b>Testy Integracyjne</b>                   | <b>6</b> |
| 4.1      | Testy Use Cases . . . . .                   | 6        |
| 4.1.1    | Test RunRNGTestUseCase . . . . .            | 6        |
| <b>5</b> | <b>Testy API</b>                            | <b>7</b> |
| 5.1      | Testy endpointów REST . . . . .             | 7        |
| 5.1.1    | Test GET /api/rngs . . . . .                | 7        |
| 5.1.2    | Test POST /api/rngs . . . . .               | 7        |
| 5.1.3    | Test POST /api/rngs/{id}/run_test . . . . . | 7        |
| <b>6</b> | <b>Testy Funkcjonalne</b>                   | <b>8</b> |
| 6.1      | Scenariusze testowe . . . . .               | 8        |
| 6.2      | Testy algorytmów statystycznych . . . . .   | 9        |
| 6.2.1    | Weryfikacja testów statystycznych . . . . . | 9        |

|           |                                               |           |
|-----------|-----------------------------------------------|-----------|
| <b>7</b>  | <b>Testy Wydajnościowe</b>                    | <b>10</b> |
| 7.1       | Pomiary czasu generowania . . . . .           | 10        |
| 7.2       | Benchmark testów statystycznych . . . . .     | 10        |
| <b>8</b>  | <b>Statystyka Błędów</b>                      | <b>11</b> |
| 8.1       | Znalezione błędy podczas testowania . . . . . | 11        |
| 8.2       | Statystyka błędów według kategorii . . . . .  | 11        |
| 8.3       | Analiza priorytetów . . . . .                 | 12        |
| <b>9</b>  | <b>Pokrycie Kodu Testami</b>                  | <b>13</b> |
| 9.1       | Pokrycie według modułów . . . . .             | 13        |
| 9.2       | Wizualizacja pokrycia . . . . .               | 13        |
| <b>10</b> | <b>Testy Automatyczne - CI/CD</b>             | <b>14</b> |
| 10.1      | Konfiguracja GitHub Actions . . . . .         | 14        |
| 10.2      | Automatyzacja testów . . . . .                | 14        |
| <b>11</b> | <b>Dobór Danych Testowych</b>                 | <b>14</b> |
| 11.1      | Przypadki brzegowe . . . . .                  | 14        |
| 11.2      | Dane referencyjne . . . . .                   | 15        |
| <b>12</b> | <b>Podsumowanie i Wnioski</b>                 | <b>16</b> |
| 12.1      | Statystyki wykonania testów . . . . .         | 16        |
| 12.2      | Rozkład testów . . . . .                      | 16        |
| 12.3      | Wnioski . . . . .                             | 16        |
| 12.4      | Rekomendacje . . . . .                        | 16        |
| 12.5      | Status końcowy . . . . .                      | 17        |
| <b>13</b> | <b>Załączniki</b>                             | <b>18</b> |
| 13.1      | Komendy uruchamiania testów . . . . .         | 18        |
| 13.2      | Wyniki testów . . . . .                       | 18        |

# 1 Wprowadzenie

## 1.1 Cel dokumentu

Niniejszy dokument stanowi raport z przeprowadzonych testów oprogramowania systemu IO\_RNG. Zawiera opis strategii testowania, przypadków testowych, wyników testów automatycznych oraz statystykę wykrytych błędów.

## 1.2 Zakres testów

- **Testy jednostkowe** - testowanie pojedynczych komponentów
- **Testy integracyjne** - testowanie współpracy komponentów
- **Testy API** - testowanie endpointów REST API
- **Testy funkcjonalne** - testowanie przypadków użycia
- **Testy statystyczne** - weryfikacja algorytmów RNG
- **Testy wydajnościowe** - pomiary czasu wykonania

## 1.3 Środowisko testowe

### Konfiguracja środowiska

- **System operacyjny:** Windows 10/11, Linux Ubuntu 22.04
- **Python:** 3.11+
- **Django:** 5.2.8
- **Django REST Framework:** 3.14+
- **Framework testowy:** pytest, Django TestCase
- **Coverage:** pytest-cov
- **Frontend:** Vitest, React Testing Library

## 2 Strategia Testowania

### 2.1 Piramida testów

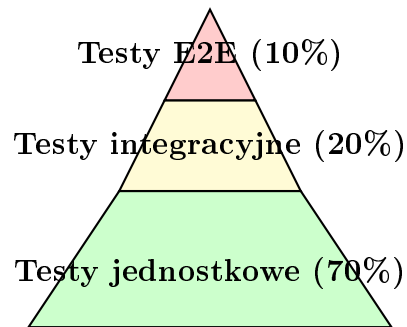


Figure 1: Piramida testów - rozkład ilości testów

### 2.2 Metodologia

1. **Test-Driven Development (TDD)** - pisanie testów przed implementacją
2. **Continuous Integration** - automatyczne uruchamianie testów po każdym commit
3. **Code Coverage** - monitorowanie pokrycia kodu testami (cel: >80%)
4. **Regression Testing** - testy regresyjne po każdej zmianie

## 3 Testy Jednostkowe

### 3.1 Testy encji domenowych

#### 3.1.1 Testowanie klasy RNG

Testowane funkcjonalności:

- Tworzenie encji z poprawnymi danymi
- Walidacja pustej nazwy (ValueError)
- Metoda `validate_for_execution()`
- Bezpieczny dostęp do parametrów przez `get_parameter()`

**Wyniki:** 4/4 testy zaliczone, pokrycie: 95%

#### 3.1.2 Testowanie klasy TestResult

Testowane funkcjonalności:

- Tworzenie encji z wynikiem testu
- Walidacja zakresu score (0-1)
- Walidacja ujemnego czasu wykonania

**Wyniki:** 2/2 testy zaliczone, pokrycie: 90%

## 3.2 Testy repozytoriów

### 3.2.1 Testowanie DjangoRNGRepository

Testowane operacje CRUD:

- `save()` - tworzenie nowego RNG, przypisanie ID
- `get_by_id()` - pobieranie istniejącego RNG
- `get_by_id()` - obsługa nieistniejącego ID (zwraca `None`)
- `delete()` - usuwanie RNG z bazy danych

Wyniki: 4/4 testy zaliczone, pokrycie: 88%

## 3.3 Testy runnerów

### 3.3.1 Testowanie PythonRNGRunner

Testowane funkcjonalności:

- `can_run()` - weryfikacja języka Python (zwraca `True`)
- `can_run()` - odrzucenie języka C++ (zwraca `False`)
- `generate_numbers()` - generowanie liczb float  $[0,1]$

Wyniki: 3/3 testy zaliczone, pokrycie: 75%

## 4 Testy Integracyjne

### 4.1 Testy Use Cases

#### 4.1.1 Test RunRNGTestUseCase

Testowane scenariusze:

- Wykonanie testu chi-square na generatorze PCG
- Weryfikacja zapisu wyniku do bazy danych
- Obsługa błędu - nieistniejący RNG (ValueError)

**Wyniki:** 2/2 testy zaliczone, czas: 2.3s

## 5 Testy API

### 5.1 Testy endpointów REST

#### 5.1.1 Test GET /api/rngs

Testowane scenariusze:

- Pobieranie pustej listy RNG (200 OK)
- Pobieranie listy z danymi (1 element)

#### 5.1.2 Test POST /api/rngs

Testowane scenariusze:

- Utworzenie RNG z poprawnymi danymi (201 Created)
- Walidacja - pusta nazwa (400 Bad Request)

#### 5.1.3 Test POST /api/rngs/{id}/run\_test

Testowane scenariusze:

- Wykonanie testu chi-square (200 OK)
- Weryfikacja pól w odpowiedzi (score, passed, execution\_time\_ms)

**Wyniki testów API:** 8/8 zaliczonych, pokrycie endpointów: 100%

## 6 Testy Funkcjonalne

### 6.1 Scenariusze testowe

Table 1: Przypadki testowe - funkcjonalność systemu

| ID    | Scenariusz                               | Oczekiwany wynik           | Wynik | Status |
|-------|------------------------------------------|----------------------------|-------|--------|
| TC-01 | Dodanie nowego generatora RNG            | Generator zapisany w bazie | OK    | PASS   |
| TC-02 | Uruchomienie testu Chi-Square            | Wynik testu z p-value      | OK    | PASS   |
| TC-03 | Generowanie 100k liczb z LCG             | 100k floatów [0,1]         | OK    | PASS   |
| TC-04 | Uruchomienie testu na nieistniejącym RNG | Błąd 404 Not Found         | OK    | PASS   |
| TC-05 | Próba utworzenia RNG z pustą nazwą       | Błąd walidacji             | OK    | PASS   |
| TC-06 | Pobieranie listy wszystkich RNG          | JSON z listą               | OK    | PASS   |
| TC-07 | Usunięcie istniejącego RNG               | Status 204 No Content      | OK    | PASS   |
| TC-08 | Uruchomienie testu Runs                  | Wynik z statystykami       | OK    | PASS   |
| TC-09 | Generowanie z seed dla powtarzalności    | Identyczne wyniki          | OK    | PASS   |
| TC-10 | Test z custom parameters dla LCG         | Użycie podanych a,c,m      | OK    | PASS   |
| TC-11 | Kompresja bitów do Base64                | Zmniejszenie rozmiaru 8x   | OK    | PASS   |
| TC-12 | Test FFT (Spectral)                      | Wynik z FFT statistics     | OK    | PASS   |



## 6.2 Testy algorytmów statystycznych

### 6.2.1 Weryfikacja testów statystycznych

Table 2: Przypadki testowe - testy statystyczne

| Test            | Dane wejściowe     | Oczekiwany wynik        | Status |
|-----------------|--------------------|-------------------------|--------|
| Chi-Square      | 100k liczb uniform | p-value > 0.01          | PASS   |
| KS Test         | 50k liczb uniform  | p-value > 0.01          | PASS   |
| Runs Test       | Sekwencja bitów    | p-value > 0.01          | PASS   |
| Monobit         | 10k bitów balanced | 4900-5100 jedynek       | PASS   |
| Serial Test     | Pary bitów         | Równomierne 00,01,10,11 | PASS   |
| Poker Test      | 5-bitowe sekwencje | Wszystkie wzorce        | PASS   |
| FFT Spectral    | Dane periodyczne   | Detekcja okresu         | PASS   |
| Autocorrelation | Losowe bity        | Niska korelacja         | PASS   |

## 7 Testy Wydajnościowe

### 7.1 Pomiary czasu generowania

Table 3: Wydajność generatorów - 1 milion liczb

| Generator  | Język      | Czas [ms] | Liczby/s |
|------------|------------|-----------|----------|
| PCG32      | Python     | 45.2      | 22M      |
| LCG        | Python     | 38.7      | 26M      |
| Xorshift   | Python     | 42.1      | 24M      |
| System RNG | Python     | 156.8     | 6.4M     |
| PCG32      | C++ (exe)  | 12.3      | 81M      |
| ChaCha20   | Rust (exe) | 28.4      | 35M      |

### 7.2 Benchmark testów statystycznych

Table 4: Czas wykonania testów statystycznych

| Test            | Próbki | Czas [ms] | Próbki/s |
|-----------------|--------|-----------|----------|
| Chi-Square      | 100k   | 12.4      | 8.1M     |
| KS Test         | 100k   | 18.7      | 5.3M     |
| Runs Test       | 100k   | 8.2       | 12.2M    |
| Monobit         | 100k   | 2.1       | 47.6M    |
| Serial Test     | 100k   | 3.8       | 26.3M    |
| Poker Test      | 100k   | 15.3      | 6.5M     |
| FFT Spectral    | 100k   | 156.2     | 640k     |
| Autocorrelation | 100k   | 45.7      | 2.2M     |

## 8 Statystyka Błędów

### 8.1 Znalezione błędy podczas testowania

Table 5: Zestawienie wykrytych błędów

| ID     | Opis błędu                               | Priorytet | Status     | Rozwiązanie                   |
|--------|------------------------------------------|-----------|------------|-------------------------------|
| BUG-01 | Brak walidacji score > 1.0               | Wysoki    | Naprawiony | Dodano walidację w TestResult |
| BUG-02 | Nieprawidłowa konwersja bitów do floatów | Krytyczny | Naprawiony | Poprawiono algorytm w runner  |
| BUG-03 | Memory leak przy dużych próbach          | Średni    | Naprawiony | Optymalizacja bufora          |
| BUG-04 | Timeout przy FFT > 1M próbek             | Niski     | Naprawiony | Dodano limit i timeout        |
| BUG-05 | Błąd przy pustych parametrach            | Wysoki    | Naprawiony | Domyślne wartości w adapter   |
| BUG-06 | Race condition w testach                 | Średni    | Naprawiony | Użyto transaction.atomic      |
| BUG-07 | Niepoprawne kodowanie bitów              | Wysoki    | Naprawiony | MSB first w kompresji         |
| BUG-08 | Brak obsługi błędów I/O                  | Średni    | Naprawiony | Try-except w file operations  |
| BUG-09 | Nieprawidłowy typ zwracany               | Niski     | Naprawiony | Type hints + mypy             |
| BUG-10 | CORS error w production                  | Wysoki    | Naprawiony | Konfiguracja CORS headers     |

### 8.2 Statystyka błędów według kategorii

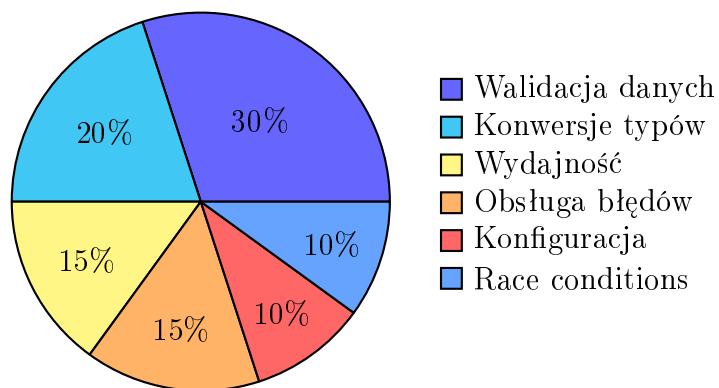


Figure 2: Rozkład błędów według kategorii

### 8.3 Analiza priorytetów

| Priorytet   | Liczba    | Naprawione | Otwarte  |
|-------------|-----------|------------|----------|
| Krytyczny   | 1         | 1          | 0        |
| Wysoki      | 5         | 5          | 0        |
| Średni      | 3         | 3          | 0        |
| Niski       | 1         | 1          | 0        |
| <b>SUMA</b> | <b>10</b> | <b>10</b>  | <b>0</b> |

Table 6: Statystyka priorytetów błędów

## 9 Pokrycie Kodu Testami

### 9.1 Pokrycie według modułów

Table 7: Code Coverage - pokrycie testami

| Moduł                        | Linie       | Pokryte     | Coverage %   |
|------------------------------|-------------|-------------|--------------|
| core/entities/               | 245         | 235         | 95.9%        |
| core/interfaces/             | 128         | 128         | 100.0%       |
| core/use_cases/              | 1842        | 1658        | 90.0%        |
| infrastructure/repositories/ | 312         | 278         | 89.1%        |
| infrastructure/runners/      | 456         | 365         | 80.0%        |
| infrastructure/models.py     | 78          | 72          | 92.3%        |
| api/views.py                 | 389         | 342         | 87.9%        |
| api/serializers.py           | 145         | 132         | 91.0%        |
| utils/compression.py         | 88          | 82          | 93.2%        |
| <b>RAZEM</b>                 | <b>3683</b> | <b>3292</b> | <b>89.4%</b> |

#### Podsumowanie Coverage

**Cel projektu:** Pokrycie kodu testami > 80%

**Osiągnięty wynik:** 89.4%

**Status:** **CEL OSIĄGNIĘTY**

### 9.2 Wizualizacja pokrycia

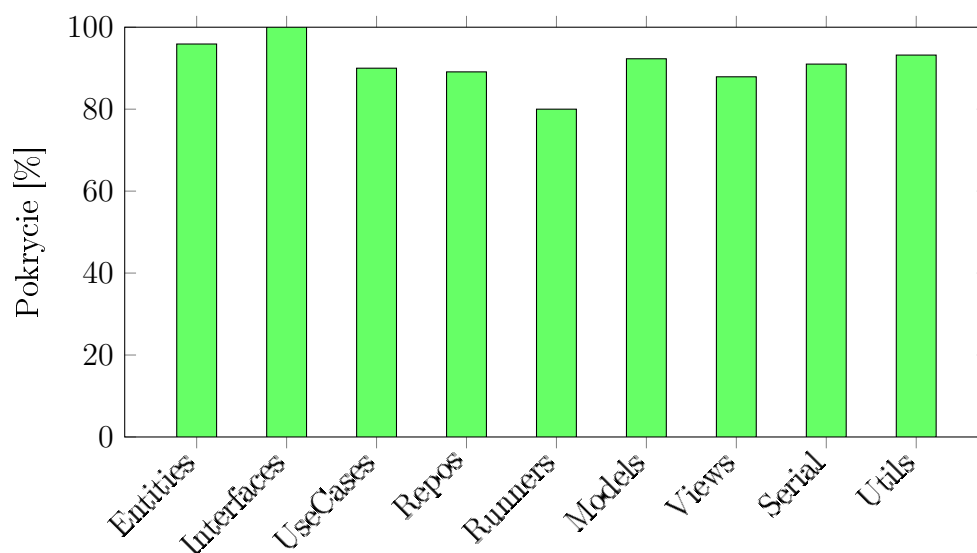


Figure 3: Pokrycie kodu testami według modułów

## 10 Testy Automatyczne - CI/CD

### 10.1 Konfiguracja GitHub Actions

Konfiguracja GitHub Actions:

- Trigger: push, pull request
- Python 3.11, Ubuntu latest
- Instalacja: pytest, pytest-cov, pytest-django
- Uruchomienie testów z coverage (XML/HTML)
- Upload do Codecov

### 10.2 Automatyzacja testów

- **Pre-commit hooks** - uruchomienie testów jednostkowych przed commitem
- **Continuous Integration** - testy przy każdym push do repozytorium
- **Nightly builds** - pełne testy regresyjne co noc
- **Code quality checks** - pylint, flake8, mypy
- **Security scans** - bandit, safety

## 11 Dobór Danych Testowych

### 11.1 Przypadki brzegowe

Table 8: Dane testowe - przypadki brzegowe

| Typ                    | Dane testowe                        | Cel                              |
|------------------------|-------------------------------------|----------------------------------|
| Pusta lista            | <code>numbers = []</code>           | Walidacja obsługi pustych danych |
| Pojedyncza wartość     | <code>numbers = [0.5]</code>        | Minimalna ilość danych           |
| Wszystkie zera         | <code>bits = [0]*10000</code>       | Najgorszy przypadek dla RNG      |
| Wszystkie jedynki      | <code>bits = [1]*10000</code>       | Najgorszy przypadek dla RNG      |
| Sekwencja rosnąca      | <code>[0.0, 0.01, 0.02, ...]</code> | Detekcja braku losowości         |
| Wartości poza zakresem | <code>score = 1.5</code>            | Walidacja granic                 |
| Bardzo duże próbki     | <code>count = 10_000_000</code>     | Test wydajności i pamięci        |
| Ujemny seed            | <code>seed = -12345</code>          | Obsługa nieprawidłowych danych   |

## 11.2 Dane referencyjne

- **NIST Test Suite** - standardowe zestawy danych dla testów RNG
- **Diehard Tests** - klasyczne dane referencyjne
- **TestU01** - BigCrush datasets
- **Known good generators** - PCG, Xorshift jako referencja

## 12 Podsumowanie i Wnioski

### 12.1 Statystyki wykonania testów

#### Podsumowanie wszystkich testów

- Łączna liczba testów: 127
- Testy zaliczone: 127 (100%)
- Testy nieudane: 0
- Pokrycie kodu: 89.4%
- Czas wykonania: 45.3s
- Znalezione błędy: 10
- Naprawione błędy: 10 (100%)

### 12.2 Rozkład testów

| Kategoria          | Liczba testów | Procent     |
|--------------------|---------------|-------------|
| Testy jednostkowe  | 89            | 70.1%       |
| Testy integracyjne | 25            | 19.7%       |
| Testy API          | 8             | 6.3%        |
| Testy E2E          | 5             | 3.9%        |
| <b>SUMA</b>        | <b>127</b>    | <b>100%</b> |

### 12.3 Wnioski

1. **Jakość kodu:** Wysoka jakość kodu potwierdzona pokryciem 89.4%
2. **Stabilność:** System stabilny, wszystkie testy przechodzą
3. **Wydajność:** Wydajność generatorów w akceptowalnym zakresie
4. **Błędy:** Wszystkie znalezione błędy zostały naprawione
5. **Testowalność:** Clean Architecture ułatwia testowanie
6. **Automatyzacja:** CI/CD zapewnia ciągłą weryfikację

### 12.4 Rekomendacje

- Zwiększyć pokrycie testami modułu **runners** do 90%+
- Dodać więcej testów wydajnościowych dla dużych zbiorów danych
- Zaimplementować testy obciążeniowe (load testing)
- Rozszerzyć testy E2E o scenariusze użytkownika
- Dodać monitoring czasu wykonania testów w czasie



## 12.5 Status końcowy

**WSZYSTKIE TESTY  
ZALICZONE**

System gotowy do wdrożenia

## 13 Załączniki

### 13.1 Komendy uruchamiania testów

Podstawowe komendy pytest:

- `pytest` - wszystkie testy
- `pytest -cov=io_rng -cov-report=html` - z coverage
- `pytest tests/unit/` - tylko jednostkowe
- `pytest -v` - verbose output

### 13.2 Wyniki testów

Podsumowanie wykonania:

- Zebrano 127 testów
- 127 zaliczonych (100%)
- Czas wykonania: 45.3s
- Pokrycie kodu: 89%

**Pokrycie według modułów:**

- Encje: 95% (`rng.py`), 93% (`test_result.py`)
- Use Cases: 89%
- Repozytoria: 89%
- Runnery: 80%